A08 项目开发规范

> 版本：v1.1
>
> 更新时间：2026-03-11
>
> 适用团队：3 人

一、开发环境规范

1.1 强制要求

项目	要求	理由
操作系统	WSL2 + Debian 11/12/13	性能最好、与 Windows 集成佳
Python 版本	3.10+（推荐 3.13）	保证语法特性一致
包管理	pip + requirements.txt	统一依赖版本
代码编辑器	VS Code + Remote WSL	团队配置统一
Git 客户端	命令行 Git	避免 GUI 工具差异

1.2 推荐配置

```
# 所有队员必须安装的 VS Code 扩展
- Remote - WSL
- Python (Microsoft)
- Pylance
- GitLens
```

1.3 环境配置（已统一）

项目	要求
操作系统	WSL2 + Debian 13
Python 版本	3.10+（推荐 3.13）
代码编辑器	VS Code + Remote WSL

二、Git 协作规范

2.1 分支命名规范（简化版）

```
main                # 主分支，保护分支，仅用于发布
dev                 # 开发分支，日常开发推送至此
```

> **注意**：MVP 阶段使用简化工作流，不需要 feature/fix 等分支，直接在 dev 开发推送。

2.2 提交信息规范

```
<类型>: <简短描述>

[可选：详细描述]

[可选：关联 Issue]
```

类型说明:

类型	说明	示例
`feat`	新功能	`feat: 添加能耗查询接口`
`fix`	修复 Bug	`fix: 修复 COP 计算错误`
`docs`	文档更新	`docs: 更新 API 文档`
`style`	代码格式	`style: 格式化代码`
`refactor`	重构	`refactor: 重构查询模块`
`test`	测试	`test: 添加单元测试`
`chore`	构建/工具	`chore: 更新依赖版本`

示例:

```
# 好的提交信息
git commit -m "feat: 添加建筑能耗查询接口"
git commit -m "fix: 修复时间范围筛选 bug"

# 不好的提交信息
git commit -m "更新代码"
git commit -m "fix bug"
git commit -m "asdfasdf"
```

2.3 Git 工作流程（简化版）

```
# 每个人都直接在 dev 分支开发

# 1. 开始开发前
git checkout dev
git pull origin dev

# 2. 开发并提交
git add .
git commit -m "feat: 添加能耗查询接口"

# 3. 直接推送到远程 dev（无需 PR）
git push origin dev
```

> **简化原则**：MVP 阶段不追求复杂流程，能跑起来最重要。main 分支仅用于最终版本发布。

2.4 Git 配置（首次使用）

```
# 所有队员必须执行
git config --global user.name "你的 GitHub 用户名"
git config --global user.email "你的 GitHub 邮箱"
git config --global core.autocrlf input  # Linux/WSL
git config --global init.defaultBranch main
```

三、代码规范

3.1 Python 代码规范

文件结构:

```
# 1. 文档字符串（模块说明）
"""
建筑能耗查询接口
"""

# 2. 导入（标准库→第三方→本地）
import os
from datetime import datetime

import pandas as pd
from fastapi import FastAPI

from .utils import helper

# 3. 常量（全大写）
DEFAULT_BUILDING_ID = "B001"

# 4. 类定义
class EnergyQuery:
    """能耗查询类"""

    def __init__(self):
        self.data = None

    def query(self, building_id: str) -> dict:
        """
        查询建筑能耗

        Args:
            building_id: 建筑编号

        Returns:
            能耗数据字典
        """
        pass

# 5. 函数定义
def calculate_cop(outlet_temp: float, return_temp: float, energy: float) ->
float:
```

```python
    """
    计算 COP 能效比

    Args:
        outlet_temp: 出水温度
        return_temp: 回水温度
        energy: 空调能耗

    Returns:
        COP 值
    """
    return (return_temp - outlet_temp) / energy

# 6. 主程序入口
if __name__ == "__main__":
    pass
```

命名规范:

类型	命名方式	示例
变量/函数	小写 + 下划线	`building_id`, `query_energy()`
类名	大驼峰	`EnergyQuery`, `DataProcessor`
常量	全大写 + 下划线	`MAX_RETRY`, `API_KEY`
私有变量	单下划线前缀	`_internal_data`

代码检查工具:

```bash
# 安装
pip install flake8 black isort

# 使用
flake8 src/              # 代码风格检查
black src/               # 自动格式化
isort src/               # 导入排序
```

3.2 前端代码规范（Streamlit）

> **技术选型**：MVP 阶段使用 Streamlit（纯 Python，无需 HTML/CSS），决赛阶段可升级为 Vue3。

目录结构:

```
src/web/                    # Streamlit 前端
├── app.py                 # 主入口
├── pages/                  # 页面目录
│   ├── query.py           # 查询页面
│   ├── stats.py           # 统计页面
│   └── qa.py              # 问答页面
├── components/             # 组件
│   ├── chart.py           # 图表组件
│   └── sidebar.py         # 侧边栏
└── utils/                 # 工具函数
    └── api_client.py      # API 调用
```

Streamlit 页面示例:

```python
import streamlit as st
import pandas as pd

st.set_page_config(page_title="建筑能源管理系统", layout="wide")

st.title("🏢 建筑能耗查询")

# 侧边栏
with st.sidebar:
    building = st.selectbox("选择建筑", ["B001", "B002", "B003"])
    date_range = st.date_input("日期范围")

# 主内容
col1, col2 = st.columns([2, 1])

with col1:
    st.subheader("能耗趋势")
    # 折线图
    st.line_chart(data)

with col2:
    st.subheader("关键指标")
    st.metric("今日用电", "1,234 kWh", "+5%")
```

四、项目目录规范

```
fwwb-a08/
├── README.md                    # 项目说明
├── requirements.txt             # Python 依赖（必须）
├── .gitignore                  # Git 忽略文件
├── .env.example                # 环境变量模板
├── docs/                       # 文档目录
│   ├── API 文档.md
│   ├── 部署手册.md
│   └── 用户手册.md
├── src/                        # 源代码
│   ├── api/                   # 后端接口
│   │   ├── __init__.py
│   │   ├── main.py           # FastAPI 入口
```

```
|   |   ├── routes/          # 路由
|   |   └── models/          # 数据模型
|   ├── web/                  # Streamlit 前端
|   |   ├── app.py           # 主入口
|   |   ├── pages/           # 页面
|   |   └── components/      # 组件
|   ├── rag/                  # RagFlow 配置
|   └── utils/                # 工具函数
├── data/                     # 数据目录
|   ├── raw/                 # 原始数据（不提交到 Git）
|   ├── processed/           # 处理后数据
|   └── dictionary.md        # 数据字典
├── tests/                    # 测试目录
|   ├── test_api.py
|   └── test_utils.py
└── scripts/                  # 脚本目录
    ├── generate_data.py      # 数据生成
    └── deploy.sh             # 部署脚本
```

五、依赖管理规范

5.1 requirements.txt 格式（暂时仅供参考）

```
# 核心依赖
fastapi==0.109.0
uvicorn[standard]==0.27.0
pydantic==2.5.0

# 数据处理
pandas==2.1.4
numpy==1.26.0

# 数据库
sqlalchemy==2.0.23

# 前端（Streamlit）
streamlit==1.30.0
plotly==5.18.0

# AI/RAG
langchain==0.1.0
chromadb==0.4.22

# 开发工具
pytest==7.4.0
black==23.12.0
flake8==7.0.0
```

5.2 依赖管理流程

```
# 1. 安装新依赖后更新 requirements.txt
pip install 新包
pip freeze > requirements.txt

# 2. 队员同步依赖
pip install -r requirements.txt

# 3. 检查依赖冲突
pip check
```

六、环境变量规范

6.1 .env.example 模板

```
# 复制为 .env 后填写真实值
cp .env.example .env

# 数据库配置
DATABASE_URL=sqlite:///./energy.db

# 大模型 API
LLM_API_KEY=your_api_key
LLM_BASE_URL=https://api.deepseek.com

# RagFlow 配置
RAGFLOW_API_KEY=your_ragflow_key
RAGFLOW_BASE_URL=http://localhost:9380

# 服务器配置
HOST=0.0.0.0
PORT=8000
DEBUG=true
```

6.2 .gitignore 配置

```
# 环境变量文件
.env
.env.local
.env.*.local

# 敏感信息
*.key
*.pem
credentials.json

# 数据文件
data/raw/*.csv
data/processed/*.db

# IDE
```

```
.vscode/
.idea/

# Python
__pycache__/
*.pyc
venv/
```

七、文档规范

7.1 必须维护的文档

文档	负责人	完成时间
README.md	队长	项目启动第 1 周
API 文档	后端开发	每个接口开发完成后
数据字典	数据负责人	数据集完成后
部署手册	技术负责人	第 6 周
用户手册	队长	第 7 周

7.2 API 文档格式

```
## 能耗查询接口

**URL**: `POST /api/query`

**请求参数**:
| 参数 | 类型 | 必填 | 说明 |
|------|------|------|------|
| building_id | string | 否 | 建筑编号 |
| start_time | datetime | 否 | 开始时间 |
| end_time | datetime | 否 | 结束时间 |

**响应示例**:
```json
{
  "code": 200,
  "data": [...],
  "total": 1000
}
```
```

八、检查清单

8.1 队员启动检查

- [ ] 配置好 WSL2 + Debian 环境
- [ ] 安装 Python 3.10+
- [ ] 安装 VS Code + 必需扩展
- [ ] 配置 Git 用户信息
- [ ] 克隆项目仓库
- [ ] 安装项目依赖
- [ ] 成功运行 Hello World

8.2 提交前检查

- [ ] 提交信息符合规范
- [ ] 已更新相关文档
- [ ] 本地测试通过

8.3 发布前检查（合并到 main 前）

- [ ] 所有功能测试通过
- [ ] 无冲突
- [ ] 队长/负责人批准

附录：快速开始指南

启动环境配置

```
# 1. 安装 WSL2（Windows）
wsl --install -d Debian

# 2. 安装 VS Code + Remote WSL 扩展

# 3. 配置 Git
git config --global user.name "你的名字"
git config --global user.email "你的邮箱"

# 4. 克隆项目
cd ~
mkdir projects && cd projects
git clone git@github.com:quetzal-china/fwwb-a08.git

# 5. 安装依赖
cd fwwb-a08
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt

# 6. 验证
```

```
python -c "print('Hello A08')"
```

如有修改建议，请在微信群及时沟通